Caterpillar

Typst content parser and custom syntax processor

Version 0.1.0

August 2026

https://github.com/retroflexivity/typst-caterpillar

Caterpillar is a package that provides tools to parse Typst content. It is primarily made for defining custom syntactic rules, but can be used for various purposes, hopefully. This is a documentation on Caterpillar.

1. Introduction

Suppose you are writing some notes for personal use. You keep them in raw text, and you care primarily about how they look in raw text. So you use syntactic conventions like this:

```
> Blah blah blah blah _blah_ blah
                              — Plato

This seems to contradict the following

Axiom 114'. Bleh bleh bleh bleh bleh.

because

a. bluh bluh
b. bluh bluh bluh
```

Suppose then that a collegue who is afraid of raw text asks you to share your notes. Or you urgently need to present something on a lab meeting. Anyway, you want to compile *this* to PDF. Given that you are reading these docs, you would probably prefer doing it with Typst.

Typst's syntactic sugar, while impressive for a PDF compiler engine, is nevertheless quite limited. It will not handle greater-than quotes, axioms, or letter-based lists. So how do you achieve a fancy-looking PDF without sacrificing your markup habits? You write a custom processor. This is what Caterpillar is for.

This package is primarily a parser engine for Typst content. You define a parser and combine it with a function (handler) that says what to do with the results of parsing, should it succeed. Caterpillar runs the parser and applies the function. Apart from this rather simple algorithm, Caterpillar provides primitives for writing parsers over content, which is sometimes more complicated than what one might expect.

This documentation consists of two parts: Section 2 is a tutorial that introduces the reader to writing parsers, and Section 3 is an API reference.

2. Tutorial

This chapter shows how to write a parser and use it for parsing custom markup syntax. We begin with the simplest case: interpreting paragraphs that start with `>` as quotes, as in the following example.

```
There once was a saying,

> Blah blah blah blah blah blah
```

2.1. Getting started

First import the package.

```
#import "@preview:caterpillar:0.1.0": *
#import parsers as p
```

There are several functions in Caterpillar. The central one is `crawl`: it takes any number of parser-handler pairs and some content, runs the parsers one by one over the content, and if a parser succeeds, applies the corresponding function to its output. It is designed to be used in a show rule, like this:

```
#show par: crawl.with()
```

Naturally, it will not do anything yet, with no parsers supplied.

The supporting function `parse` is there to simply run a parser over content and obtain the result.

The `parsers` module, which we abbreviate as `p`, contains some predefined parsers and parser combinators — functions used for building complex parsers from simpler ones.

2.2. Parsers

A Caterpillar parser is a function that takes an array of contents and returns a **match result** — a dictionary with keys `matched`, `match`, and `rest`.

- `matched` is a boolean, telling whether the parser has succeeded.
- `match` is the part of the content that corresponds to the parser — the content that the parser has *consumed*. Its type is determined by the parser — a content, an array, or `none`.
- `rest` is the part of the content that the parser has not consumed. It is an array of contents.

Thus, `match` and `rest` combined roughly correspond to the input.

Conceptually, a parser defines conditions on content: what can be parsed. A parser goes through contents from left to right. If the parser stumbles across something that does not correspond to its condition, it fails, returning a match result with `matched: false`. If the beginning of contents corresponds to the parser's condition, it returns a match result with `matched: true`.

To grasp the idea of how parsers work, imagine we have a parser that parses one or more `a`'s from a string (in regex, we would write `a+`).

- Given a string `aaaaa`, the parser will succeed, consuming it all.
- Given a string `aaaabaa`, the parser will succeed, too, consuming the first `aaaa`. It will not go further, because it does not parse `b`'s, so `baa` will stay as the rest.
- Given a string `baaaa`, the parser will fail: going left to right, it will bump into a `b` that it cannot parse, and will not satisfy its requirement that there be at least one `a`.

There are several kinds of functions in the `parsers` submodule. First, there are constants: unflexible parsers that parse certain hard-coded values. For example, there is a constant `space`, which parses spaces, and a constant `end`, which only succeeds when there is nothing more to parse.

Second, there are primitive parser operators that handle values of certain types. They are two-place curried functions: they accept a value that tells them what they should parse and by this means become parsers (functions from content to match results). There is a parser `string` that defines how to search content for strings, and `exact` that defines how to search content for other content. In your parsers, you do not have to write `string` or `exact`: you just pass strings or contents, and Caterpillar figures out what to do itself.

Third, there are parser combinators. They take other parsers and modify them in a certain way. `multiple` is the simplest combinator, running several parsers one after another[1]. Another example, `repeat`, runs a parser a specified or unspecified number of times (it does the job of regex's `*`, `+`, and `{x,y}`).

2.3. Inside the content

Let us look at what the content we want to parse consists of. Typst exposes convenient methods `fields` to understand the structure of contents and `func` to see its type.

[1] You can also just pass an array.

```
#let c = [> Blah blah blah blah blah blah]
a #c.func() with fields #c.fields()
```

---

a text with fields (text: "> Blah blah blah blah blah blah")

This line consists of a single piece of text-type content. A text is Typst's default textual structure with one field, eponymously named `text`, of type `string`. But let us look at something more complex.

```
#let c = [> _Blah_ blah blah blah blah blah]
a #c.func() with fields #c.fields()
```

```
a sequence with fields (
  children: (
    [>],
    [ ],
    emph(body: [Blah]),
    [ ],
    [blah blah blah blah blah],
  ),
)
```

When content contains something other than text, it becomes a multipart structure of type `sequence`. A sequence has children, which are pieces of content, too. The first child of our sequence is still a text, but now it only includes the >. The space has got separated!

```
#let c = [> _Blah_ blah blah blah blah blah]
a #c.children.at(0).func() and a #c.children.at(1).func()
```

---

a text and a space

When a single space is surrounded by text, Typst joins them together for performance considerations. However, in all other cases, such as when there is a non-text content on one of the sides, the space becomes a separate `space` element. The same happens if you happen to type a double space: it is because Typst fixes double spaces for you.

```
#let c = [>  Blah blah blah blah blah blah]
#c.fields()
```

```
(
  children: ([>], [ ], [Blah blah blah blah blah blah]),
)
```

Caterpillar is designed to take care of these inconsistencies by itself. The aforementioned `space` parser parses both the space element and the string space. Caterpillar also handles the difference between a `text` and a `sequence`, so the same parser can parse both simple and complex contents.

This brief introduction to Typst's content structure is enough to write our first parser.

2.4. Writing the parser

As a reminder, we need to write a parser that will parse the second paragraph in the text below, extracting the greater-then sign and the space. It should also handle the variations mentioned in Section 2.3.

```
There once was a saying,

> Blah blah blah blah blah blah
```

Our parser, thus, will consist of two parts: a greater-than sign parser and the `space` parser. Parsing a piece of text is done with the `exact` parser.[2] To chain multiple parsers, we use the `multiple` combinator: it runs parsers one by one, each parser taking the previos one's rest, and fails if any of its subparsers fails.

```
#let parser = p.multiple(p.exact[>], p.space)

#parse(parser)[> Blah blah blah blah blah blah]
```

```
(
  matched: true,
  match: ([>], [ ]),
  rest: ([Blah blah blah blah blah blah],),
)
```

The parser has succeeded: the prefix is in the `match` fields, and the quotation body is in `rest`. When given a text that is longer than what the `exact` parser should parse, it gets the string out of it and tries comparing prefixally: thus, it extracts the `">"` from the text `"> Blah blah blah blah blah blah"`. Let us verify this works when the space is a separate content:

```
#let parser = p.multiple(p.exact[>], p.space)

#parse(parser)[> _Blah_ blah blah blah blah blah]
```

```
(
  matched: true,
  match: ([>], [ ]),
  rest: (
    emph(body: [Blah]),
    [ ],
    [blah blah blah blah blah],
  ),
)
```

Note the types of the return values. `match` is an array, because `multiple` produces an array. Other parsers produce other types: as you can see, the outputs of `exact` and `space` inside the array are of type `content`. Parsers that do not match anything by design produce `none`. You can always consult the API in Section 3 or via the language server: the type declaration is written in the end of the description. E.g. `multiple` is `..parser ⇒ array(content) ⇒ match-result(array)`, meaning that it takes any number of parsers, then the content to run the parsers on (it is always an array in Caterpillar internally — this is handled by `parse`) and returns a `match-result` where `match` is of type `array`.

`rest` is also an array. This is because Caterpillar works on arrays instead of sequences for simplicity. `crawl` then joins the rest, creating a content — you can try it yourself with `rest.join()`.

[2] `string` can also be used here with no real difference.

As mentioned above, Caterpillar exposes some syntactic sugar to let you avoid writing trivial `multiple` and `exact`. Arrays are passed to `multiple`, and content to `exact`. Cleaning up the parser:

```
#let parser = ([>], p.space)

#parse(parser)[> Blah blah blah blah blah blah]
```

```
(
  matched: true,
  match: ([>], [ ]),
  rest: ([Blah blah blah blah blah blah],),
)
```

What if we run the parser on something that is not a quote? Let us do a sanity check.

```
#let parser = ([>], p.space)

#parse(parser)[There once was a saying,]

#parse(parser)[>5 experts validated this.]
```

```
(
  matched: false,
  match: none,
  rest: ([There once was a saying,],),
)
(
  matched: false,
  match: ([>],),
  rest: ([5 experts validated this.],),
)
```

Neither matches, which is exactly what we need — no false positives meaning no unexpected quotes in the resulting document.

2.5. Using the parse results

Having written the parser, we can now make Caterpillar turn the match results into actual quotes. For this, we use a show rule with `crawl`.

```
#let parser = ([>], p.space)
#let handler(match, rest) = quote(rest, block: true)
#show par: crawl.with(
  (parser, handler)
)

There once was a saying,

> Blah blah blah blah blah blah
```

There once was a saying,

> Blah blah blah blah blah blah

`crawl` needs to be run on paragraphs: a paragraph is the structure that gets parsed. We pass to it a parser-handler pair. The handler is a two-place function: if the parser succeeds, `crawl` runs it with the `match` and `rest` (the latter gets joined) from the match result.

2.6. A more complex example

So far so good. Now let us try to also get the attribution of the quote, as in the following.

```
> Blah blah blah blah _blah_ blah
                             — Plato
```

First we inspect it.

```
#let c = [> Blah blah blah blah _blah_ blah
                             — Plato]
#c.fields()
```

```
(
  children: (
    [> Blah blah blah blah],
    [ ],
    emph(body: [blah]),
    [ ],
    [blah],
    [ ],
    [–],
    [ ],
    [Plato],
  ),
)
```

As you can see, Typst joined the newline and spaces into a single `space` element. While it takes away some flexibility, we can still handle this.

Let us first write a helper parser for the attribution.

```
#let attr-parser = (space, [—], space, repeat(anything, min: 1))
```

The parser will consume the newline and the tabulation, and then the attribution marker[3]. `anything` is a constant that parses a single piece of content — any content. This way, we can include anything we want in an attribution, including non-textual elements. `repeat` will repeat `anything` any number of times, but at least one.

What we want the parser to do is parse the content *until* it encounters an attribution, then stop and parse the rest as an attribution. Luckily, there is a combinator `until` precisely for this purpose.

[3]Note that two dashes are interpreted as an en dash by the Typst compiler. But so do they in the parser, which means an em dash will match against an en dash.

```
#let attr-parser = (p.space, [—], p.space, p.repeat(p.anything, min: 1))

#let parser = (
  [>], p.space,
  p.until(p.anything, attr-parser),
  attr-parser
)
```

`until` takes two parsers — read `until(a, b)` as "parse *a* until you can parse *b*". It first tries to parse *b*. If parsing *b* succeeds, it stops immediately, without consuming *b*. If it fails, it parses *a* and repeats (or, if parsing *a* fails, fails as a whole).

Here, *b* is complex: it contains multiple parsers and even another parser combinator. It is never a problem for Caterpillar: parsers can be nested as deeply as needed.

Running the parser:

```
#let attr-parser = (p.space, [—], p.space, p.repeat(p.anything, min: 1))

#let parser = (
  [>], p.space,
  p.until(p.anything, attr-parser),
  attr-parser
)

#parse(parser)[> Blah blah blah blah _blah_ blah
                            — Plato]
```

```
(
  matched: true,
  match: (
    [>],
    [ ],
    (
      [Blah blah blah blah],
      [ ],
      emph(body: [blah]),
      [ ],
      [blah],
    ),
    ([ ], [-], [ ], ([Plato],)),
  ),
  rest: (),
)
```

Everything we need is now in `match` instead of `rest` (we parse to the end with `repeat(anything)`). The third item is always the body of the citation, and the fourth item of the fourth item is the attribution.

Let us see if the parser works on attribution-less quotes.

```
#let attr-parser = (p.space, [—], p.space, p.repeat(p.anything, min: 1))

#let parser = (
  [>], p.space,
  p.until(p.anything, attr-parser),
  attr-parser
)

#parse(parser)[> Blah blah blah blah _blah_ blah]
```

```
(
  matched: false,
  match: ([>], [ ]),
  rest: (
    [Blah blah blah blah],
    [ ],
    emph(body: [blah]),
    [ ],
    [blah],
  ),
)
```

Unfortunately, it does not: it parses everything but then breaks. This is because `until` never encounters its second parser, hitting the end of content. There are two ways to avoid this issue: either allow `until` to parse the end as well, or make the whole piece that was added in this section optional. We will proceed with option 1: implementing option 2 should be easy after reading the following section.

2.6.1. Disjunction and optionality

`end` is a special constant that matches empty content: that is, the content that has come to its end. We want `until` to stop either on the attr-parser or at the end. Disjunctive parsing is done by the `one-of` combinator.

The other piece that we need is `optional`. It attempts to parse, but doesn't break if it fails: in that case, it simply parses nothing. We need it to prevent `attr-parser` from breaking the parse after `until` finishes at the end[4].

[4] A cleaner implementation would actually repeat `p.one-of(attr-parser, p.end)` instead, but I need to demonstrate the behavior of `optional`.

```
#let attr-parser = (p.space, [—], p.space, p.repeat(p.anything, min: 1))
#let parser = (
  [>], p.space,
  p.until(p.anything, p.one-of(attr-parser, p.end)),
  p.optional(attr-parser)
)

#parse(parser)[> Blah blah blah blah _blah_ blah
                            — Plato]

#parse(parser)[> Blah blah blah blah _blah_ blah]
```

```
(
  matched: true,
  match: (
    [>],
    [ ],
    (
      [Blah blah blah blah],
      [ ],
      emph(body: [blah]),
      [ ],
      [blah],
    ),
    ([ ], [-], [ ], ([Plato],)),
  ),
  rest: (),
)
(
  matched: true,
  match: (
    [>],
    [ ],
    (
      [Blah blah blah blah],
      [ ],
      emph(body: [blah]),
      [ ],
      [blah],
    ),
    none,
  ),
  rest: (),
)
```

The parser now succeeds on both kinds of quotes, returning `none` if no attribution is present. It is now time to add a handler.

```
#let attr-parser = (p.space, [—], p.space, p.repeat(p.anything, min: 1))
#let parser = (
  [>], p.space,
  p.until(p.anything, p.one-of(attr-parser, p.end)),
  p.optional(attr-parser)
)
#let handler(match, rest) = {
  let attr = if match.at(3) != none {match.at(3).at(3).join()} else {none}
  quote(
    match.at(2).join(),
    attribution: attr,
    block: true
  )
}
#show par: crawl.with(
  (parser, handler)
)

Plato once said,

> Blah blah blah blah _blah_ blah
                            — Plato

Someone else once said,

> Blah blah blah blah _blah_ blah
```

Plato once said,

> Blah blah blah blah *blah* blah
>
> — Plato

Someone else once said,

> Blah blah blah blah *blah* blah

2.7. Conclusion

This concludes the tutorial. The material given here should be sufficient for parsing all other syntax given in Section 1, as well as most of what you might need. For more information on parsers, consult the API reference below.

3. API reference

3.1. Primary API

3.1.1. parse

Run a parser on content. Split content automatically.

`parser, content => match-result`

Parameters

```
parse(
  p: parser,
  c: content
)
```

3.1.2. crawl

Parse a paragraph (or any content) trying every parser, and apply the corresponding handler to the first success, otherwise keep the paragraph as is.

`..(parser, handler), content => content`
where `handler := (any, content) => content`

Parameters

```
crawl(
  ..prefs: args(parser, handler),
  par: content
)
```

..prefs `args(parser, handler)`

Any number of pairs (parser, handler), where the handler takes the match and the rest (joined automatically) and produces content from them.

par `content`

Either a simple content (sequence) or anything that has fields `body` (like `par`).

3.2. Parsers

3.2.1. multiple

Sequentially run a list of parsers.

Tip. This has syntactic sugar: just pass an array of parsers. `parse(multiple(p, q, r))[...]` = `parse((p, q, r))[...]`

`..parser => array(content) => match-result(array)`

Example

```
#parse(([more], p.space, [than]))[more
than one]
```

```
(
  matched: true,
  match: ([more], [ ], [than]),
  rest: ([ one],),
)
```

Parameters

`multiple(..ps: args(parser))`

..ps `args(parser)`

Any number of parsers.

3.2.2. test

Fail if the parser fails, otherwise succeed without consuming.

`parser => array(content) => match-result(none)`

Example

```
#parse(p.test[test])[testing is not
particularly exciting]
```

```
(
  matched: true,
  match: none,
  rest: ([testing is not particularly exciting],),
)
```

Parameters

`test(p: parser)`

3.2.3. test-not

Fail if the parser succeeds, otherwise succeed without consuming.

`parser => array(content) => match-result(none)`

Example

```
#parse(p.test-not[not test])[testing is
not particularly exciting]
```

```
(
  matched: true,
  match: none,
  rest: ([testing is not particularly exciting],),
)
```

Parameters

`test-not(p: parser)`

3.2.4. optional

Run a parser optionally, returning none instead of failing when not matched.

```
parser => array(content) => match-result(content | none)
```

Example

```
#parse(p.optional[actually])[nothing
actual there]
```

```
(
  matched: true,
  match: none,
  rest: ([nothing actual there],),
)
```

Parameters

```
optional(p)
```

3.2.5. one-of

Try every parser until one succeeds.

```
..parser => array(content) => match-result(any)
```

Example

```
#parse(p.one-of([first], [second],
[third]))[second guess]
```

```
(matched: true, match: [second], rest: ([ guess],))
```

Parameters

```
one-of(..ps: args(parser))
```

..ps `args(parser)`

Any number of parsers.

3.2.6. all

Run all the parsers on the same content. If all succeed, return the result of the first.

```
..parser => array(content) => match-result(any)
```

Example

```
#parse(p.all(p.text, [part]))[part of the
whole]
```

```
(
  matched: true,
  match: [part of the whole],
  rest: (),
)
```

Parameters

```
all(..ps: args(parser))
```

..ps `args(parser)`

Any number of parsers.

3.2.7. repeat

Run the parser multiple times, with the minimum and the maximum number of times specified by `min` and `max` parameters. If `max` ≤ 0, repeat ad infinum. If `min` > `max`, `min` is ignored.

```
(parser, min: int, max: int) => match-result(array)
```

Examples

```
#parse(p.repeat(("very", p.space)))[very very very many repetitions]
```

```
(
  matched: true,
  match: (([very], [ ]), ([very], [ ]), ([very], [ ])),
  rest: ([many repetitions],),
)
```

```
#parse(p.repeat(("very", p.space), max: 2))[very very very many repetitions]
```

```
(
  matched: true,
  match: (([very], [ ]), ([very], [ ])),
  rest: ([very many repetitions],),
)
```

```
#parse(p.repeat(("very", p.space), min: 4))[very very very many repetitions]
```

```
(
  matched: false,
  match: (([very], [ ]), ([very], [ ]), ([very], [ ])),
  rest: ([many repetitions],),
)
```

Parameters

```
repeat(
  p: parser,
  min: int,
  max: int
)
```

min `int`

Minimum number of times to repeat. Will fail if not reached.

Default: `0`

max `int`

Maximum number of times to repeat. Will stop when reached. -1 means unlimited.

Default: `-1`

3.2.8. until

Continuously run the first parser until the second succeeds. Does not consume the match of the second parser.

`(parser, parser) => array(content) => match-result(array)`

Example

```
#parse(p.until(p.anything, [...]))[wait
wait wait... you can go]
```

```
(
  matched: true,
  match: ([wait wait wait],),
  rest: ([...], [ ], [you can go]),
)
```

Parameters

```
until(
  p: parser,
  end: parser
)
```

p `parser`

Parser to repeat.

end `parser`

Parser to check and stop if it succeeds.

3.2.9. predicate

Parse a single piece of content that corresponds to a predicate.

`(content => bool) => array(content) => match-result(content)`

Example

```
#parse(p.predicate(it => it.func() ==
box))[#box[a box] and text]
```

```
(
  matched: true,
  match: box(body: [a box]),
  rest: ([ ], [and text]),
)
```

Parameters

```
predicate(p: function)
```

p `function`

A predicate on a single piece of content.

3.2.10. string

Parse the text that matches the parser string or regex exactly from the beginning of the contents.

TIP. This has syntactic sugar: just pass a string or a regex. `parse(string("..."))[...] == parse("...")[...]`, `parse(string(regex(".*")))[...] == parse(regex(".*"))[...]`

`str | regex => array(content) => match-result(content)`

Example

```
#parse(regex("\d+"))[15mm]
```

```
(matched: true, match: [15], rest: ([mm],))
```

Parameters

`string(p: str regex)`

p `str` or `regex`

A string or a regex.

3.2.11. exact

Parse possibly multipart content that matches the parser content exactly or begins with it.

TIP. This has syntactic sugar: just pass pure content. `parse(exact[...])[...] == parse([...])[...]`

`content => array(content) => match-result(content)`

Examples

```
#parse([but is it])[but is it a proper example?]
```

```
(
  matched: true,
  match: [but is it],
  rest: ([ a proper example?],),
)
```

```
#parse([but _is_ it])[but _is_ it a proper example?]
```

```
(
  matched: true,
  match: sequence([but], [ ], emph(body: [is]), [ ], [it]),
  rest: ([ a proper example?],),
)
```

Parameters

`exact(p: content)`

p `content`

A content, possibly complex.

3.2.12. anything

Any piece of content. Defined as `predicate(_ => true)`.

`array(content) => match-result(content)`

Example

```
#parse(p.anything)[_*well*_, anyway]
```

```
(
  matched: true,
  match: emph(body: strong(body: [well])),
  rest: ([, anyway],),
)
```

3.2.13. text

Any piece of content of type `text`. Defined as `predicate(c => c.func() == text)`.

`array(content) => match-result(content)`

Example

```
#parse(p.text)[might _probably_ turn out
useful.]
```

```
(
  matched: true,
  match: [might],
  rest: (
    [ ],
    emph(body: [probably]),
    [ ],
    [turn out useful.],
  ),
)
```

3.2.14. space

Either a space element or a " " string. Defined as `one-of([ ], " ")`

`array(content) => match-result(content)`

Example

```
#parse(("a", p.space))[a text with spaces]
```

```
(
  matched: true,
  match: ([a], [ ]),
  rest: ([text with spaces],),
)
```

```
#parse(p.space)[  a so-called tabulation]
```

```
(
  matched: true,
  match: [ ],
  rest: ([a so-called tabulation],),
)
```

3.2.15. end

Empty content.

`array(content) ⇒ match-result(none)`

Example

```
#parse(p.end)[]
```

```
(matched: true, match: none, rest: ())
```

3.3. Utility

3.3.1. is-match-result

match-result schema. a match result is a dictionary with fields

```
match-result := (
  matched :: bool
  match :: any
  match :: array(content)
)
```

Parameters

`is-match-result(v)`

3.3.2. is-parser

parser schema. A parser is a singleton expression that gets compared to the content.

```
parser :=
    content
  | string
  | content ⇒ match-result
  | array(parser)
```

Parameters

`is-parser(v)`

3.3.3. run-parser

A low-level parser runner. Curried: turns a parser (polymorphic) into a parser function, to be run on an array of contents.

`function | content ⇒ function`

Parameters

`run-parser(p)`